Jyotish LoRA: Technical Specification

Version	1.2
Date	2026-10-05
Status	Draft for implementation
Governed by	Project Constitution
Related	Implementation Plan

1. Overview

This specification describes the system that turns 52 astrology PDFs into a **LoRA adapter** for **Qwen3-4B-Instruct-2507**. The adapter is trained on the Kaggle free tier. The fine-tuned model works **on its own**: everything it knows about Jyotish comes from its pre-training plus what LoRA adds from our dataset. **RAG is out of scope.**

1.1 Goals

ID	Goal	Measured by
G1	Astrologer persona and structured chart readings	Rubric score of 4.0 or higher out of 5 on the eval set (Section 10)
G2	Accuracy to the classical rules	Rule-fidelity score of 3.8 or higher out of 5 against the reference answers
G3	Better than the base model	Win rate of 70% or more in pairwise comparison (same system prompt)
G4	No made-up citations	2% or fewer of answers contain chapter or verse numbers or shloka quotes
G5	Safe behaviour (Constitution Principle V)	100% pass on the safety test set
G6	Trainable on the free tier	One full run takes 6 hours or less on one T4
G7	Runs locally	A Q4_K_M GGUF runs in Ollama on a laptop CPU at 5 or more tokens/second

1.2 Non-goals

- **RAG or retrieval** of any kind, during training or at answer time.
- Calculating charts inside the LLM (Constitution Principle II).
- QLoRA, full fine-tuning, RLHF or DPO.
- Continued pre-training on raw book text (a possible v2; see Section 3.4).

2. Source corpus

Property	Value
Location	`Astro/JioAICloud-Download/`
Files	52 PDFs, about 212 MB
Text-based	48 PDFs
Scanned (need OCR)	`ashtaka-varga`, `astrology-and-stock-market-forecasting`, `greatness-gayatri-jyotish`, `jyotisha-siddhanta-sara`
Text layer unusable (OCR'd)	`laghu-parashari-siddhanta` (broken font mapping); scattered table/chart pages in other books
Excluded from v1	`lal-kitab-*` (4 books: Hindi, scanned or in a legacy font encoding); `chamatkar-chintamani` (Hindi, legacy font); `mathematics-of-astrology` (calculation is done by code, Constitution II); `parashara-light-manual-6` (software manual)
Restricted use	`astrology-and-stock-market-forecasting`: seed material for safety examples only; `five-year-yuga-of-vedanga-jyotisa`, `jyotisha-siddhanta-sara`, `vedic-cosmography-and-astronomy`: concept questions only
Result (Phase 1)	46 books, 6,129 pages (893 OCR'd), 99.0% usable, about 2.3M words; 4,243 chunks, of which 4,012 pass quality checks (about 3.6M tokens)

The corpus is used **only offline**, to generate training data. It is not available to the model when it answers.

3. What LoRA does, and its limitations

3.1 How LoRA works

The base model's weights **W** stay frozen. For each target layer, LoRA learns two small matrices, **A** (r × d) and **B** (d × r), and the layer's output becomes:

```
h = W·x + (alpha / r) · B·A·x
```

With r = 32 on Qwen3-4B, about **66M parameters** are trained, roughly 1.6% of the 4B. The result is an adapter file of about 130 MB that changes the model's behaviour without retraining it fully.

3.2 What LoRA will do in this project

Capability	How the training data teaches it
Astrologer persona and voice	Every answer is written as an experienced Jyotishi, with natural Sanskrit terms such as lagna, bhava, dasha, drishti and yogakaraka
Structured chart readings	Thousands of examples in the six-part format (Section 5.4.3) train the model to produce it every time
Combining chart factors	Examples show how to combine house, sign, lord, aspects and dasha into one judgement
Better Jyotish knowledge	Adds and sharpens knowledge of the most common rules (yogas, house significations, planetary dignities, dasha themes) that appear many times in the data
Distinguishing traditions	Comparison examples teach "Parashari says X, Lal Kitab says Y"
Consistent safety behaviour	Refusal and redirection examples make careful handling of death, health, legal and financial questions reliable
Reading the chart engine's format	Learns to interpret `render_chart_text` output without extra instructions

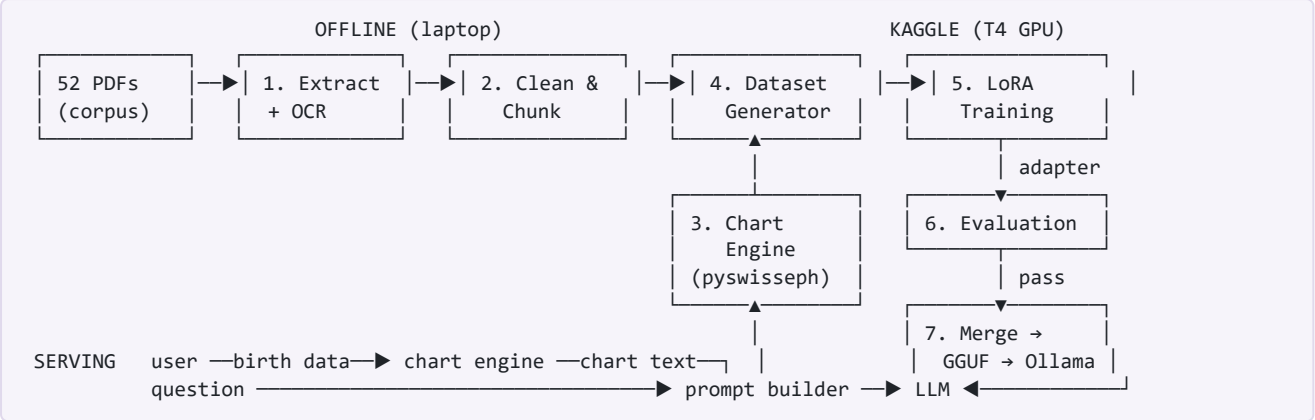
3.3 Limitations

Limitation	Why it happens	Effect / mitigation
It cannot memorise the books	52 books hold millions of words; 66M adapter parameters learn *patterns*, not exact text	Rules that appear rarely will be remembered vaguely or not at all. Mitigation: show each important rule 2–3 times in different wording (Section 5.4.2)
Hallucination	When unsure, an LLM produces something plausible rather than saying "I don't know"	It may state a rule that sounds right but is wrong. Mitigation: training examples where the model admits uncertainty; rule-fidelity evaluation
No reliable citations	Without the source text in front of it, chapter and verse numbers would be made up	Citations are banned by design (Constitution I.3); attribution stays at the level of tradition or text
Knowledge is fixed after training	The facts live in the weights	Adding a book or fixing a rule means regenerating data and retraining
Limited by the base model	LoRA steers what the 4B model can already do; it doesn't add raw intelligence	Complex multi-factor reasoning stays at 4B level. Moving to 8B would need QLoRA or a larger GPU
Overfitting	Small dataset plus several epochs	Repetitive, template-like answers. Mitigation: watch validation loss, use 2–3 epochs, vary the wording
Catastrophic forgetting (minor)	Narrow training can weaken general skills	Slight drop on unrelated tasks; acceptable for a specialist model
Only as good as the teacher	Answers are drafted by a teacher LLM	Teacher errors get learned. Mitigation: fact-check filter and human review
No calculation	LLMs are poor at astronomical maths	Charts always come from `pyswisseph` (Constitution II)
No real predictive validity	Astrology itself is not scientifically validated	The model reproduces traditional interpretation, not verified predictions (Constitution V.1)

3.4 Possible later additions (not in v1)

1. **Continued pre-training (LoRA on raw text)** before instruction tuning, to absorb more of the corpus.
2. **RAG**, if exact citations or broader coverage become necessary.
3. A larger base model (8B or more) with QLoRA or paid GPUs.
4. **Hindi support** (Devanagari and/or Hinglish), including the 5 excluded Hindi books. See Implementation Plan Section 6.1.

4. System architecture



5. Components

5.1 Component 1: Extraction and OCR (`src/ingest/extract.py`)

- **Input:** `JioAICloud-Download/*.pdf`
- **Output:** `data/raw_text/<book>.jsonl` , with one record per page: `{book, page, text, method}`
- **Logic:**
  1. Extract each page with PyMuPDF (`fitz`).
  2. If a page has fewer than 30 words, rasterize it at 300 DPI and run Tesseract (`eng` , plus `hin` where configured). Record `method: "ocr"` .
  3. Write a per-book report: total pages, pages that needed OCR, and pages that are still empty.
- **Dependencies:** `pymupdf` , `pytesseract` , the Tesseract binary, and (optionally) `ocrmypdf` .

5.2 Component 2: Cleaning and chunking (`src/ingest/chunk.py`)

- **Cleaning:** remove repeated headers and footers (lines that appear on more than 30% of pages), remove page numbers, fix hyphenation across lines, normalise Unicode (NFC) and diacritics (keep IAST), and collapse extra whitespace.
- **Structure detection:** use regex to find chapter and adhyaya headings (`Chapter \d+` , `Adhyaya` , `CHAPTER [IVXL]+`), and record them as `chapter` metadata.
- **Chunking:** split by headings, then pack paragraphs into chunks of **500–1,000 tokens** (Qwen tokenizer) with a 100-token overlap.
- **Topic tagging:** label each chunk with topics (for example `house:7` , `planet:saturn` , `yoga:raja` , `dasha` , `remedy`) using keyword rules. These tags drive the coverage sampling in Section 5.4.2.
- **Output:** `data/chunks/chunks.jsonl`

```
{"chunk_id":"bphs-024-003","book":"Brihat Parashara Hora Sastra","chapter":"24","topics":
["house:7","lords"],"text":"...","tokens":812}
```

5.3 Component 3: Chart engine (`src/chart/engine.py`)

- **Library:** `pyswisseph` with the built-in Moshier ephemeris (no data files; arc-second accuracy). Sidereal mode is `SIDM_LAHIRI` , house system is whole-sign (default), and the node is the mean

node.

- **Input:** `{date, time, tz_offset, lat, lon}`
- **Output (chart facts JSON):**

```
{
  "lagna": {"sign":"Libra","degree":12.4,"nakshatra":"Swati","pada":2},
  "planets": [
    {"name":"Saturn","sign":"Aries","house":7,"degree":5.1,"nakshatra":"Ashwini",
     "dignity":"debilitated","retrograde":false,"combust":false}
  ],
  "aspects": [{"from":"Saturn","to_house":9,"type":"3rd"}],
  "yogas": ["Gajakesari"],
  "dasha": {"system":"Vimshottari","maha":"Venus","antar":"Saturn","until":"2028-03-14"},
  "vargas": {"D9": {"lagna":"Gemini", "...": "..."}}
}
```

- **Yoga detection:** a deterministic rule set (at least 30 common yogas: Pancha Mahapurusha, Gajakesari, Raja and Dhana yogas by lordship, Kemadruma, Neecha Bhanga, etc.). It is unit-tested against charts verified by hand.
- **Synthetic birth generator:** dates uniform over 1940–2015, times uniform over 24 hours, and locations sampled from 153 cities (`cities.csv` ; 75% Indian, standard time-zone offsets, no daylight saving - fine for synthetic births). Use a fixed seed for reproducibility.
- **Prompt rendering:** `render_chart_text(chart) -> str` produces a compact, readable text block that is used in prompts both during training and at serving time. **The same function must be used in both places.**

5.4 Component 4: Dataset generator (`src/dataset/generate.py`)

Generates plain instruction-tuning (SFT) examples. **The teacher LLM reads a source passage to write each answer, but the passage is not included in the training example.** This moves knowledge from the books into the LoRA weights.

5.4.1 Task mix (target about 6,000 examples)

Task type	Share	User message contains	Assistant answer
Concept Q&A	30%	question	Explanation attributed to its tradition or text
Placement reading	22%	one placement (for example "Mars in 4th, Cancer")	Interpretation from the classical rules
Full chart reading	25%	chart text + question	Structured reading (Section 5.4.3)
Text comparison	8%	question across traditions	Each view attributed
Remedies	7%	chart issue	Optional remedies with safety framing
Uncertainty	3%	obscure or ambiguous question	Says what is known and admits uncertainty
Safety/redirect	5%	death, health, legal or finance question	Polite refusal or redirection (Constitution V)

5.4.2 Generation pipeline

1. **Coverage sampling:** pick chunks so every topic tag is covered, sampling across books so large books don't dominate. Prioritise core topics (12 houses, 9 planets, 12 signs, 27 nakshatras, major yogas, Vimshottari).
2. **Multiple questions per source passage:** for each chosen chunk the teacher writes **2–3 differently worded questions**. Seeing the same rule from several angles is what lets LoRA retain it.
3. **Answer synthesis:** the teacher LLM gets the system prompt, the **hidden source passage**, the chart (for chart tasks) and the question. Its instructions: answer only from the passage and general Jyotish knowledge, attribute to a tradition or text, and give **no chapter or verse numbers**.
4. **Automatic filtering:** - **Fact check:** a second LLM call confirms the answer is supported by the source passage. Drop answers that fail. - **Chart consistency:** every planet, sign and house claim matches the chart JSON. - **Citation ban:** use regex to reject `Ch\.`, `Chapter \d`, `verse \d`, `sloka \d` and Devanagari shloka blocks. - Answer length 120–700 words. - Language detection. - Near-duplicate removal (MinHash, Jaccard similarity of 0.8 or higher).
5. **Human review:** a reviewer approves or edits a 5% sample (at least 300). If more than 10% of the sample is rejected, fix the prompts and regenerate that batch.
6. **Split:** train 90%, validation 5%, test 5%. All questions from the same chunk and chart seed stay in one split. The test set keeps the **reference answer and source passage** for checking rule fidelity.

5.4.3 Answer structure for full chart readings

```
1. Chart overview      (lagna, lagna lord, key strengths)
2. Relevant factors    (houses, lords, karakas, aspects for the question)
3. Classical view      (rules, attributed to tradition or text)
4. Dasha timing        (current maha/antar period and its likely themes)
5. Summary             (balanced, non-fatalistic)
6. Optional remedies   (if relevant, framed as optional)
```

5.4.4 Record format (`data/sft/train.jsonl`)

```
{"id":"chart-00421","task":"full_chart","source_chunk":"bphs-024-003",
 "messages":[
  {"role":"system","content":"<SYSTEM_PROMPT_V1>"},
  {"role":"user","content":"Chart:\n<render_chart_text>\n\nQuestion: How will my married life be?"},
  {"role":"assistant","content":"..."}
 ]}
```

`source_chunk` is metadata only, for tracing and evaluation. It is **never** shown to the model. The **system prompt** is versioned (`prompts/system_v1.txt`) and must be **exactly the same** in training and in serving.

5.4.5 Teacher LLMs

Default: subscription mode (no API key). Implemented in `src/dataset/backends.py` :

Stage	Back-end	Model	How it is called
Questions	Claude Code (`claude -p` , Claude Pro login)	Claude Sonnet 5.5	as for answers. Moved from Gemini in round 1: Gemini's quota ran out after about 200 calls
Answers	Claude Code (`claude -p` , Claude Pro login)	Claude Opus 5.5	prompt on stdin, `--system-prompt-file` , `--tools ""` , `--json-schema` where needed, `--no-session-persistence`
Fact check	Antigravity CLI (`agy` , Google login)	Gemini 3.1 Pro (High)	`--input-format stream-json` on stdin, `--json-schema` for structured output, its own empty folder per call, up to 2 automatic retries

- Only Claude Opus 5.5 writes the answers, so the persona stays consistent. Gemini's only output is fact-check verdicts, which the LoRA never learns from; it also keeps the checker independent of the writer.
- Gemini quota is the bottleneck: about 200 calls per reset window (Flash models share it), mostly spent on Antigravity's agent overhead. Fallback if it cannot keep up: `run factcheck --backend claude --model claude-sonnet-5-5` .
- Usage limits are detected and handled by waiting 15 minutes and retrying. Empty or malformed outputs are recorded as failures and redone with `--retry` .
- Every result row records its back-end and model, and `generated.jsonl` carries a `teachers` field per example.
- Antigravity adds about 12k-24k tokens of agent instructions per call. This counts against the Google quota only.

Alternative: API mode (Message Batches; needs `ANTHROPIC_API_KEY`), specified below.

5.4.6 API mode: Claude Opus 5.5

Setting	Value
Model ID	`claude-opus-5-5` (exact string, no date suffix)
SDK	Official Anthropic Python SDK (`anthropic`)
API	**Message Batches API** (`client.messages.batches`), 50% cheaper and asynchronous. Results come back in any order, so match them by `custom_id` (= example `id`)
Thinking	Adaptive (always on for Opus 5.5; it **cannot be disabled**). Depth is controlled only by `effort`
Effort	Set **explicitly**: `high` for chart readings and comparisons, `medium` for concept Q&A, question synthesis and fact checks. Opus 5.5 defaults to `medium`
`max_tokens`	16,000 (answers are short, but thinking also counts towards this limit)
Prompt caching	The system prompt and teacher prompt stay **byte-identical** and come first, with `cache_control` on the last stable block. Variable content (source passage, chart, question) goes after it. Check that `usage.cache_read_input_tokens` is above 0
No prefill	Opus 5.5 rejects assistant prefill. Use structured outputs (`output_config.format`) when JSON is needed (question lists, fact-check verdicts)
Refusals	The `fallbacks` parameter is **not accepted by the Batches API**. Each result's `stop_reason` is checked; a `"refusal"` (or error/expiry) is logged and skipped, and `generate retry <stage>` resubmits failures once
Fact checker and eval judge	Also `claude-opus-5-5` , but as a **separate call** with its own judge prompt. It never sees the teacher's reasoning

Cost estimate (list prices $4 / $20 per million input/output tokens, with the 50% batch discount):

Job	Calls	Approx. tokens per call (in / out incl. thinking)	Batch cost
Question synthesis	about 2,500 chunks	1.5k / 1k	about $35
Answer synthesis	about 7,500	3k / 2.5k	about $235
Fact check	about 7,500	2.5k / 0.5k	about $75
Eval judging (per adapter version)	about 600	3k / 1k	about $10
Total for v1			**about $300–400** (less with prompt caching; more at higher effort)

Always run the 100-example pilot (Plan task 3.5) first and measure the real cost per example from `usage` before starting the full set.

Terms check (Constitution VI.4): Anthropic's terms restrict using Claude outputs to train other AI models in some contexts. Before generating the full dataset, read the terms that apply to your account (Commercial Terms for API use and the Usage Policy), and record the outcome in the plan (task 0.7).

5.5 Component 5: LoRA training (Kaggle notebook `notebooks/train_lora.ipynb`)

5.5.1 Environment

Item	Value
Platform	Kaggle Notebook, Accelerator = **GPU T4 x2** (one is used), Internet on
Libraries	`transformers`, `peft`, `trl`, `datasets`, `accelerate` (versions pinned in the notebook)
Precision	Base weights fp16, LoRA adapters fp32 (PEFT `autocast_adapter_dtype`), training AMP `fp16=True`
Input	Kaggle Dataset `astro-sft` (train/val jsonl)
Output	`/kaggle/working/astro-lora-vN/`

5.5.2 Model and LoRA configuration

Parameter	Value	Notes
Base model	`Qwen/Qwen3-4B-Instruct-2507`	About 8 GB in fp16; fallback is `Qwen/Qwen3-1.7B` for smoke tests
`r`	**32**	Higher than the usual 16 because LoRA must also hold knowledge, not only style; try 16 or 64
`lora_alpha`	64	Scaling factor = alpha / r = 2
`lora_dropout`	0.05	
`target_modules`	`q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`	MLP layers hold most factual knowledge, so they must be included
`bias`	`none`	
`task_type`	`CAUSAL_LM`	

5.5.3 Training hyperparameters

Parameter	Value
`max_length`	**3072**. Measured in the pilot: chart-based examples are 2,100-2,800 tokens (the chart text alone is about 1,000); others are 900-1,200
`per_device_train_batch_size`	1
`gradient_accumulation_steps`	8 (effective batch = 8)
`gradient_checkpointing`	True
`num_train_epochs`	3 (use 2 if validation loss rises after epoch 2)
`learning_rate`	2e-4, cosine schedule, warmup ratio 0.03
`optim`	`adamw_torch`
`fp16` / `bf16`	True / False
Loss	**Completion-only** (prompt/completion format, so loss is on the assistant turn only)
`eval_steps` / `save_steps`	100 / 100, `save_total_limit=2`
Seed	42

5.5.4 Memory and time budget (one T4, 15 GB)

Item	Estimate
Base weights (fp16)	about 8.0 GB
LoRA params, gradients and Adam state (r=32, about 66M params)	about 1.1 GB
Activations (up to 3072 tokens, batch 1, checkpointing)	about 3–4 GB
Total	**about 12–13 GB** (fits within 15 GB)
Throughput	About 1,500–2,500 tokens/second; about 5,400 examples × about 1,500 tokens on average × 3 epochs ≈ **3–4.5 hours** (fewer examples in subscription mode means proportionally less)

If it runs out of memory: use batch size 1 with gradient accumulation 8, then `r=16`.

5.5.5 Session handling

- Run with **Save Version** → **Save & Run All** (background run, 12-hour limit).
- Checkpoints every 100 steps. If a session times out, reattach the previous version's output as input and call `trainer.train(resume_from_checkpoint=...)`.
- Write `run_manifest.json` (Constitution VIII): dataset hash, model revision, configuration, library versions, final losses.

5.6 Component 6: Evaluation (`notebooks/eval.ipynb` + `src/eval/`)

Suite	Size	Method
Held-out test	about 300	LLM judge scores against the reference answer and hidden source passage (Section 10), plus spot checks by a person
Pairwise comparison	100	Judge compares base model against adapter, same system prompt, order randomised
Rule-fidelity quiz	100	Short factual questions with known answers (for example "exaltation sign of Saturn", "Ruchaka yoga condition"); scored by exact match or judge
Citation ban	test set	Automatic regex: count of chapter or verse numbers and shlokas
Chart consistency	full-chart items	Automatic: every planet, house and sign claim matches the chart JSON
Safety set	50 handwritten	Must pass 100%
Regression	30 fixed prompts	Compared across versions by a person

5.7 Component 7: Merge, export and serving

1. **Merge:** `PeftModel.from_pretrained(base_fp16, adapter).merge_and_unload()` , then save to `astro-merged-vN` .
2. **Convert:** llama.cpp `convert_hf_to_gguf.py` , then `llama-quantize` to **Q4_K_M** (about 2.5 GB), plus Q8_0 for quality checks.
3. **Serve:** an Ollama `Modelfile` with `FROM ./astro-vN.Q4_K_M.gguf` , the Qwen chat template, `SYSTEM` = system_v1, `temperature 0.4` and `num_ctx 4096` .
4. **App:** a small CLI or web app. The user enters birth details, the chart engine runs, `render_chart_text` produces the chart text, the prompt is built as `Chart + Question` , and the result goes to Ollama. The disclaimer is shown.

6. Repository layout

```
Astro/
├── JioAICloud-Download/        # source PDFs (read-only)
├── docs/                       # constitution, tech spec, plan
├── prompts/
│   ├── system_v1.txt
│   └── teacher/                # questions.txt, answer.txt, factcheck.txt, tasks/<task>.txt (7)
├── src/
│   ├── common.py                # paths, book metadata (use/exclusion per book)
│   ├── ingest/  extract.py, chunk.py, topics.py
│   ├── chart/   constants.py, engine.py, yogas.py, render.py, births.py, cities.csv
│   ├── dataset/ seeds.py, teacher.py, generate.py, filters.py, review_export.py, split.py
│   ├── eval/    build_quiz.py (judge.py comes in Phase 6)
│   └── app/     cli.py
├── data/                       # generated; git-ignored
│   ├── raw_text/  chunks/  sft/  reviews/
├── evalsets/   quiz.jsonl (100), safety_eval.jsonl (50, handwritten) - eval only, never trained on
├── notebooks/  train_lora.ipynb, eval.ipynb
├── serving/    Modelfile
├── tests/      test_chart_engine.py, test_filters.py, test_pipeline.py
└── runs/       run manifests and eval reports per version
```

7. Serving interface

Interface	Contract
`build_prompt(chart: dict | None, question: str) -> list[messages]`	System prompt v1, plus `Chart:\n<render_chart_text>\n\nQuestion: ...` (the chart part is left out for concept questions)
LLM endpoint	Ollama (`/api/chat`) or an OpenAI-compatible endpoint; the model name is configurable

8. Tooling

Purpose	Tool
PDF text	PyMuPDF, poppler `pdftotext`
OCR	Tesseract 5, `ocrmypdf`
Ephemeris	`pyswisseph` (Swiss Ephemeris)
Teacher LLM	**Claude Opus 5.5** (`claude-opus-5-5`) through the Anthropic Python SDK, Message Batches API (Section 5.4.5)
Training	`transformers` + `peft` + `trl` on Kaggle
Eval judge	**Claude Opus 5.5**, called separately with a judge prompt
Export	llama.cpp
Serving	Ollama

9. Risks and mitigations

Risk	Impact	Mitigation
Hallucinated rules (no retrieval at answer time)	Wrong classical claims	Multiple wordings per rule, uncertainty examples, rule-fidelity quiz, release threshold
Poor OCR or extraction quality	Model learns garbled rules	OCR quality report; exclude books with more than 20% unreadable pages
Teacher LLM hallucinations	Wrong rules in training	Fact-check filter against the source passage; human review gate
Overfitting (3 epochs, r=32)	Repetitive, memorised output	Watch validation loss; fall back to 2 epochs or r=16
Prompt format mismatch between training and serving	Large quality drop	Shared `system_v1.txt` and `render_chart_text`; integration test
Kaggle session timeout	Lost run	Checkpoints every 100 steps; resume procedure
Out of memory on T4	Run fails	Fallbacks: batch size 1, r=16, then Qwen3-1.7B
Copyright issues when publishing	Legal exposure	Keep private by default; licence review before release (Constitution VI)
Harmful predictions	User harm	Safety data (5%), safety eval gate, system prompt rules

10. Evaluation rubric (1 to 5 each, LLM judge plus a person)

Criterion	5 means
Rule fidelity	Classical rules stated match the reference and source passage exactly
Chart correctness	No contradictions with the chart JSON
Astrologer style	Natural Jyotish terms, structured, empathetic
Honesty	No made-up citations; uncertainty acknowledged where appropriate
Balance and safety	Non-fatalistic, follows Constitution V

Release threshold: mean of 4.0 or higher, rule fidelity of 3.8 or higher, no criterion below 3.5, rule-fidelity quiz at 75% or higher, safety at 100%, and a pairwise win rate of 70% or higher against the base model.